Temco Platform

The local assistant and its MCP server · built Nov 10, 2026

The local assistant and its MCP server

The assistant runs a model on the user's machine. This document specifies the tool server it calls, the protocol between them, the code on both sides, the build that produces the server, and the way the two processes find each other inside the packaged application.

It is organised in three parts:

1. **What exists** — the current call path, measured, with file and line references (§1–§2).
2. **What will be built** — the C# server and the TypeScript client, in full (§4–§6).
3. **Build and run** — how the server is compiled, where it is installed, and how it is started from inside the running application (§7–§8).

Everything in §3 marked *proven* was compiled and executed while this document was written — the C# server on .NET 10.0.400 with `ModelContextProtocol` 2.3.1, the TypeScript client typechecked with `tsc` 5 against `@modelcontextprotocol/client` 2.4.0 and the declarations in `src/vscode-dts/vscode.d.ts`. The probe project is described in §3.4 so the result can be reproduced.

1. What runs today

One funnel. Every AI feature in the product ends at `complete()`.

TEXT

```
webview (temcoAi.panel)                aiPage.ts
   │  postMessage {type:'ask', text}
   ▼
pipeline                               ai/panel.ts
   │  retrieve(docs) -> prompt -> complete()
   ▼
complete(context, messages, options)   ai/runtime.ts:104
   │  POST http://127.0.0.1:<port>/v1/chat/completions
   ▼
llama-server                           spawned at ai/runtime.ts:144
   │  --model <gguf> --host 127.0.0.1 --port <n> --ctx-size 8192
   ▼
Qwen2.5-Instruct (1.5B / 3B / 7B, Q4_K_M)   pinned in ai/catalog.ts
```

The request body is built at `ai/runtime.ts:115`:

JSON

```
{ "messages": [ ... ], "temperature": 0.2, "stream": false, "max_tokens": 640 }
```

There is no `tools` key, the answer is read as a single string, and `llama-server` is started without `--jinja`. The assistant answers from retrieved text; it cannot read a point, query the catalog, or run a program.

The state it already keeps, which the server reuses rather than replaces:

Thing	Where	Shape
Runtime and model pins	`extensions/temco-ai/src/ai/catalog.ts`	llama.cpp `b11577` per platform, Qwen2.5 1.5B/3B/7B; `ArtifactKind = 'runtime' | 'model'`, entries carry `platform`, `architecture`, `sha256`
Download, checksum, unpack	`extensions/temco-ai/src/ai/install.ts`	`install(context, entry)`, `installedArtifacts(context)`, `modelRoot(context)`
The install root	`install.ts:41`	`temcoAi.modelFolder`, else `context.globalStorageUri.fsPath`; each artifact unpacks into a folder named `entry.id`
The one HTTP call	`ai/runtime.ts:104`	`complete()`
Prompt versions	`ai/prompts.ts`	`ai-v5`
Citation logging	`ai/store.ts`	`temcocontrols.temco-store` API
Catalog database	`extensions/temco-store/src/catalog/store.ts:70`	`globalStorage/catalog/catalog.db`, `node:sqlite`, WAL

2. What the model could not do, and what changed

Missing piece	Consequence	State
`--jinja` on the `llama-server` argument list	The server never applies the model's chat template, so it never emits `tool_calls` and rejects a `tools` payload	done
`tools` in the request body	The model is never told a tool exists	done
Reading `choices[0].message.tool_calls`	A reply that asks for a tool is treated as final text	done
Something that executes a call	The extension host could, but then the tool surface is private to the assistant	done: `tools/temco-mcp`
A loop over `tool_calls`	One turn is one request, so a call cannot be answered	done: `ai/agent.ts`
A service for the tools to ask	The device, repository and project tools need one HTTP surface	not built (open decision 1)
The release step that publishes the server	There is nothing to download until a release carries it	done: section 7.4

3. What is proven, and what is not

> This section is the record of the first end-to-end run, over `stdio`. The application no longer uses that transport: it starts the server in `http` mode and manages the process (§6.5, §8.4). The transcripts below are kept because they are what established that the server, the tool list and the schema are right — and because the `stdio` mode still exists and still behaves this way.

3.1 The server, and the client, over stdio

Built, published, and driven over stdio. Transcript from the actual run:

TEXT

```
-> {"jsonrpc":"2.0","id":1,"method":"initialize","params":{"protocolVersion":"2025-06-18
<- {"result":{"protocolVersion":"2025-06-18","capabilities":{"logging":{},"tools":{"list

-> {"jsonrpc":"2.0","method":"notifications/initialized"}

-> {"jsonrpc":"2.0","id":2,"method":"tools/list","params":{}}
<- {"result":{"tools":[{"name":"device.list","title":"List devices","description":"Every

-> {"jsonrpc":"2.0","id":3,"method":"tools/call","params":{"name":"device.read_point","a
<- {"result":{"content":[{"type":"text","text":"pointType must be INPUT, OUTPUT or VARIA
```

What that establishes:

- The attributes on a plain class, discovered by `WithToolsFromAssembly()`, produce a `tools/list` whose `inputSchema` is generated from the C# signature, and whose `annotations` carry `readOnlyHint`, `destructiveHint`, `idempotentHint` and `openWorldHint`. The extension keeps its own approval list anyway (section 6.4): a server is not the authority on whether its tools are safe.
- A parameter with a default (`string? nameContains = null`) has `"default": null` and is absent from `required`; without the default it is `["string","null"]` listed as required.
- A tool that returns `CallToolResult` with `IsError = true` sends its message to the client verbatim. A tool that throws does not: the same call once threw `ArgumentException` and the client received only `"An error occurred invoking 'device.read_point'."`. That is why `ToolResults.Guard` exists and why no tool throws.

The whole round trip was then run through the compiled extension code (`out/ai/mcp.js`, plain Node, no editor) against the built server, with a stub platform service on `127.0.0.1:3210`:

TEXT

```
log [mcp] 23 tools from .../temco-mcp.exe
tools: 23, first: device.list readOnly=true
device.list -> [{"Serial":1234,"Name":"AHU-1","ProductType":"T3-BB","Address":"192.168.1
               {"Serial":1235,"Name":"VAV-2","ProductType":"T3-LB","Address":"192.168.1
catalog.tables -> no catalog database is available on this machine
bad argument -> pointType must be INPUT, OUTPUT or VARIABLE, not 'NOPE'.
disposed
```

3.2 The tool list, as built

Twenty-three tools, each with the hints that describe it:

Tool	Reads / writes
`device.list`, `device.describe`, `device.read_point`	reads
`device.trendlog_list`, `device.trendlog_query`	reads
`device.alarm_list`	reads
`device.write_point`, `device.alarm_acknowledge`	writes
`docs.search`, `docs.read`, `docs.outline`, `docs.stale`	reads
`repo.list`, `repo.show`, `repo.status`, `repo.log`, `repo.diff`	reads
`project.list`	reads
`project.create`	writes
`catalog.tables`, `catalog.query`	reads, read-only connection
`learn.search`, `learn.post_status`	reads

Fifty is the ceiling in practice: a local 7B model loses accuracy as the list grows and the arguments deepen, so every argument here is a scalar.

3.3 The extension half, and what remains unproven

#	Link	Status
1	The extension host may spawn a child process	proven: `ai/runtime.ts` already spawns `llama-server`
2	The client loads in a CommonJS extension with no dependency	proven: `ai/mcp.ts` requires only `child_process` and `fs`; the compiled `out/ai/mcp.js` ran the round trip above
3	A downloaded executable runs from the install root	proven by the same mechanism the runtime uses: `install()` unpacks into `modelRoot(context)` and records `artifact.path`
4	Two processes can share the catalog	proven: `catalog/store.ts` sets `journal_mode = WAL`; the server opens the file `ReadOnly` with a `busy_timeout`
5	The server never writes next to itself	proven: state goes to `TEMCO_MCP_STATE` and the catalog
6	`llama-server` emits `tool_calls` when started with `--jinja`	vendor-documented; the runtime is not installed on this machine. Section 8.5 step 2 settles it
7	A turn completes end to end inside the app	needs 6, a model, and the platform service. Section 8.5 step 3
8	The packaged application runs the installed server	follows from rows 1 and 3, but has not been run from an installed build

3.4 Findings the implementation now depends on

- **`Microsoft.Extensions.Hosting` is a required explicit reference.** `ModelContextProtocol` does not flow the assembly that contains `Host.CreateApplicationBuilder`; without it the build fails with `CS0103: The name 'Host' does not exist in the current context`.
- **`Microsoft.Data.Sqlite` 10.0.0 pulls a flagged native SQLite.** Restore raises `NU1903` (high severity, `SQLitePCLRaw.lib.e_sqlite3` 2.1.11). 10.0.14 restores and builds with zero warnings.
- **A tool with an optional parameter must default the token too.** C# requires optional parameters after required ones, so a tool ends with `(..., string? optional = null, CancellationToken cancellationToken = default)`, or the compiler rejects the order with `CS1737`.
- **stdin must stay open.** Driving the server from a file that ends immediately produced a full log, executed tool calls, and zero responses: the transport saw end-of-input and finished.

4. Change 1 — let the runtime call tools

4.1 The argument list

TYPESCRIPT

```typescript
  const server = spawn(
    runtime.path,
    [
      '--model',
      model.path,
      '--host',
      '127.0.0.1',
      '--port',
      String(port),
      '--ctx-size',
      '8192',
      // Without this the server never applies the model's chat template, so a `tools` p
      // rejected and a reply never carries `tool_calls`.
      '--jinja',
    ],
    { windowsHide: true }
  );
```

4.2 The types

TYPESCRIPT

```typescript
/** One tool as the model sees it: a name, a sentence, and a JSON Schema for the argumen
export interface ToolSpec {
  name: string;
  description: string;
  parameters: Record<string, unknown>;
}

/** One call the model asked for, answered by whoever owns the tool. */
export interface ToolCall {
  id: string;
  name: string;
  arguments: Record<string, unknown>;
}

export interface Completion {
  text: string;
  toolCalls: readonly ToolCall[];
}
```

`ChatMessage`, at `ai/runtime.ts:38`, gains the third role that carries a tool result and the two fields that pair a call with its answer:

TYPESCRIPT

```typescript
export interface ChatMessage {
  role: 'system' | 'user' | 'assistant' | 'tool';
  content: string;
  tool_calls?: { id: string; type: 'function'; function: { name: string; arguments: stri
  tool_call_id?: string;
}
```

4.3 `complete()` gains `tools` and returns the calls

TYPESCRIPT

```typescript
export async function complete(
  context: vscode.ExtensionContext,
  messages: readonly ChatMessage[],
  options: {
    onToken?: (text: string) => void;
    token?: vscode.CancellationToken;
    maxTokens?: number;
    /** Omitted for the plain question-and-answer path, which stays exactly as it is. */
    tools?: readonly ToolSpec[];
  } = {}
): Promise<Completion> {
  const server = await startAiServer(context);
  const payload = JSON.stringify({
    messages,
    temperature: 0.2,
    stream: options.onToken !== undefined,
    max_tokens: options.maxTokens ?? DEFAULT_MAX_TOKENS,
    ...(options.tools?.length
      ? {
          tools: options.tools.map((tool) => ({
            type: 'function',
            function: { name: tool.name, description: tool.description, parameters: tool
          })),
          tool_choice: 'auto',
          parallel_tool_calls: true,
        }
      : {}),
  });
  const response = await post(server.port, '/v1/chat/completions', payload, options.onTc
  touchIdle();
  return readCompletion(response, options.onToken !== undefined);
}

/**
 * A streamed turn is text-only: a half-rendered call cannot be answered, so a turn that
 * never streamed.
 */
function readCompletion(raw: string, streamed: boolean): Completion {
  if (streamed) {
    return { text: raw, toolCalls: [] };
```

```
  }
  const parsed = JSON.parse(raw) as {
    choices?: {
      message?: {
        content?: string | null;
        tool_calls?: { id: string; function: { name: string; arguments: string } }[];
      };
    }[];
  };
  const message = parsed.choices?.[0]?.message;
  const toolCalls = (message?.tool_calls ?? []).map((call) => ({
    id: call.id,
    name: call.function.name,
    arguments: parseArguments(call.function.arguments),
  }));
  return { text: message?.content ?? '', toolCalls };
}

function parseArguments(text: string): Record<string, unknown> {
  try {
    const value: unknown = JSON.parse(text || '{}');
    return typeof value === 'object' && value !== null ? (value as Record<string, unknow
  } catch {
    return {};
  }
}
```

`complete()` changes shape, so all four call sites read `.text` instead of the string: `ai/panel.ts:219`, `ai/panel.ts:308`, `extension.ts:128`, `extension.ts:140`. With `tools` omitted they behave exactly as they do now.

---

5. The server — `tools/temco-mcp`

5.1 Layout

TEXT

```
tools/temco-mcp/
  Temco.Mcp.csproj
  Program.cs                host, logging to stderr, stdio and http transports
  Platform/
    IPlatformApi.cs         the outside world, as one interface
    HttpPlatformApi.cs      the implementation: base URL from the environment
  Tools/
    DeviceTools.cs          [McpServerToolType]
    DocsTools.cs
    RepoTools.cs
    CatalogTools.cs
  Store/
    CatalogDb.cs            read-only SQLite over the extension's catalog
    Audit.cs                one row per call
  tests/
    Temco.Mcp.Tests.csproj  drives the real host over an in-memory transport
```

It lives at the repository root rather than under `extensions/` because it is not an extension: it is a program the extension starts.

5.2 `Temco.Mcp.csproj`

XML

```xml
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <InvariantGlobalization>true</InvariantGlobalization>
    <SatelliteResourceLanguages>en</SatelliteResourceLanguages>
    <!-- A trimmed build breaks WithToolsFromAssembly(); a single-file build extracts e_
         time. The publish output is a plain folder instead. -->
    <PublishSingleFile>false</PublishSingleFile>
    <PublishTrimmed>false</PublishTrimmed>
    <AssemblyName>temco-mcp</AssemblyName>
    <RootNamespace>Temco.Mcp</RootNamespace>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="ModelContextProtocol" Version="2.3.1" />
    <!-- Explicit: ModelContextProtocol does not flow the assembly that declares Host. -
    <PackageReference Include="Microsoft.Extensions.Hosting" Version="10.0.14" />
    <!-- 10.0.0 restores SQLitePCLRaw.lib.e_sqlite3 2.1.11, which NuGet flags NU1903 (hi
    <PackageReference Include="Microsoft.Data.Sqlite" Version="10.0.14" />
  </ItemGroup>
</Project>
```

5.3 `Program.cs`

CSHARP

```csharp
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using Microsoft.Extensions.Logging;
using Temco.Mcp;
using Temco.Mcp.Tools;

var builder = Host.CreateApplicationBuilder(args);

// stdout is the protocol. Every log line must go to stderr, or the first write breaks t
builder.Logging.ClearProviders();
builder.Logging.AddConsole(options => options.LogToStandardErrorThreshold = LogLevel.Tra

builder.Services.AddSingleton<IPlatformApi>(_ => new HttpPlatformApi(
    Environment.GetEnvironmentVariable("TEMCO_PLATFORM_API") ?? "http://127.0.0.1:3210",
    TimeSpan.FromSeconds(20)));
builder.Services.AddSingleton(_ => new Audit(
    Environment.GetEnvironmentVariable("TEMCO_MCP_STATE")
        ?? Path.Combine(Path.GetTempPath(), "temco-mcp")));

builder.Services
    .AddMcpServer(options =>
    {
        options.ServerInfo = new() { Name = "temco-mcp", Version = "1.0.0" };
    })
    .WithStdioServerTransport()
    .WithToolsFromAssembly();

await builder.Build().RunAsync();
```

The two `using Temco.Mcp…` lines are load-bearing: top-level statements sit in the global namespace, so the project's own types are not in scope without them.

5.4 A tool, in full

The attribute defaults are the unsafe ones — `ReadOnly = false`, `Destructive = true`, `OpenWorld = true` — so every tool states its own intent.

CSHARP

```
using System.ComponentModel;
using System.Text.Json;
using ModelContextProtocol.Protocol;
using ModelContextProtocol.Server;

namespace Temco.Mcp.Tools;

[McpServerToolType]
public sealed class DeviceTools(IPlatformApi platform, Audit audit)
{
    [McpServerTool(
        Name = "device.list",
        Title = "List devices",
        ReadOnly = true,
        Destructive = false,
        Idempotent = true,
        OpenWorld = false)]
    [Description("Every T3000 device known to the panel: serial, name, product type, add
    public async Task<CallToolResult> ListDevices(
        [Description("Optional text; only devices whose name contains it are returned.")
        CancellationToken cancellationToken = default)
    {
        try
        {
            var devices = await platform.GetDevicesAsync(nameContains, cancellationToken
            audit.Record("device.list", JsonSerializer.Serialize(new { nameContains }),
            return Text(JsonSerializer.Serialize(devices));
        }
        catch (Exception error) when (error is HttpRequestException or InvalidOperationE
        {
            return Fail($"the platform service did not answer: {error.Message}");
        }
    }

    [McpServerTool(
        Name = "device.read_point",
        Title = "Read a point",
        ReadOnly = true,
        Destructive = false,
        Idempotent = true,
```

```
    OpenWorld = false)]
[Description("Read one input, output or variable of a device: its value, units, and
public async Task<CallToolResult> ReadPoint(
    [Description("Device serial number.")] int serial,
    [Description("Point type: INPUT, OUTPUT or VARIABLE.")] string pointType,
    [Description("One-based point index, as the point list shows it.")] int pointInd
    CancellationToken cancellationToken = default)
{
    if (pointType is not ("INPUT" or "OUTPUT" or "VARIABLE"))
    {
        return Fail($"pointType must be INPUT, OUTPUT or VARIABLE, not '{pointType}'
    }
    try
    {
        var point = await platform.ReadPointAsync(serial, pointType, pointIndex, can
        audit.Record("device.read_point", JsonSerializer.Serialize(new { serial, poi
        return Text(JsonSerializer.Serialize(point));
    }
    catch (Exception error) when (error is HttpRequestException or InvalidOperationE
    {
        return Fail($"device {serial} has no {pointType} point {pointIndex}: {error.
    }
}

[McpServerTool(
    Name = "device.write_point",
    Title = "Write a point",
    ReadOnly = false,
    Destructive = true,
    Idempotent = false,
    OpenWorld = true)]
[Description("Write one point. Changes equipment state, so it is never called withou
public async Task<CallToolResult> WritePoint(
    [Description("Device serial number.")] int serial,
    [Description("Point type: OUTPUT or VARIABLE.")] string pointType,
    [Description("One-based point index.")] int pointIndex,
    [Description("The value to write.")] double value,
    CancellationToken cancellationToken = default)
{
    try
    {
```

```
            var result = await platform.WritePointAsync(serial, pointType, pointIndex, v
            audit.Record("device.write_point", JsonSerializer.Serialize(new { serial, pc
            return Text(JsonSerializer.Serialize(result));
        }
        catch (Exception error) when (error is HttpRequestException or InvalidOperationE
        {
            return Fail($"the write did not reach device {serial}: {error.Message}");
        }
    }

    private static CallToolResult Text(string text) =>
        new() { Content = [new TextContentBlock { Text = text }] };

    private static CallToolResult Fail(string message) =>
        new() { IsError = true, Content = [new TextContentBlock { Text = message }] };
}
```

Three rules follow from §3.1 and hold for every tool:

1. **Return `CallToolResult`; do not throw.** An exception reaches the model as `"An error occurred invoking '<tool>'."` with no reason, so a model that passed a wrong argument cannot correct it. That includes infrastructure failures: a refused connection to the platform service is the tool's error to report, not an exception to leak.
2. **Give an optional parameter a default, and default the token with it.** `string? nameContains = null` yields `"default": null` and keeps the name out of `required`; without it the schema lists it as required. Because C# requires optional parameters to come after required ones, a tool with any optional argument ends its list with `(… , string? optional = null, CancellationToken cancellationToken = default)` — the token needs the default too, or the compiler rejects the order (`CS1737`).
3. **State the four hints on every tool.** `ReadOnly`, `Destructive`, `Idempotent`, `OpenWorld` default to `false`, `true`, `false` and `true` respectively, and reach the client as `annotations`; leaving them out claims the tool destroys things and reaches outside the machine.

As built, the two result constructors and the guard live in `Tools/ToolResults.cs` so that no tool repeats them: `Text(value)`, `Fail(message)` and `Guard(body)`, which turns a refused connection, a missing record or a cancellation into a sentence. The device tools are typed because their answers have a shape; the read-only rest go through `IPlatformApi.ReadJsonAsync(path)` and return the service's own JSON rather than growing a method per route.

5.5 The platform interface

The tools do not speak T3000 protocols. They call one interface, so the transport can change without touching a tool.

CSHARP

```csharp
namespace Temco.Mcp;

public interface IPlatformApi
{
    Task<IReadOnlyList<DeviceInfo>> GetDevicesAsync(string? nameContains, CancellationTc
    Task<PointInfo> ReadPointAsync(int serial, string pointType, int pointIndex, Cancell
    Task<WriteResult> WritePointAsync(int serial, string pointType, int pointIndex, doub
}

public sealed record DeviceInfo(int Serial, string Name, string ProductType, string? Adc
public sealed record PointInfo(string Label, double Value, string Units, string Descript
public sealed record WriteResult(bool Accepted, string Message);
```

CSHARP

```csharp
using System.Net.Http.Json;
using System.Text.Json;

namespace Temco.Mcp;

public sealed class HttpPlatformApi : IPlatformApi
{
    private readonly HttpClient _http;
    private static readonly JsonSerializerOptions Json = new(JsonSerializerDefaults.Web)

    public HttpPlatformApi(string baseUrl, TimeSpan timeout)
    {
        _http = new HttpClient { BaseAddress = new Uri(baseUrl), Timeout = timeout };
    }

    public async Task<IReadOnlyList<DeviceInfo>> GetDevicesAsync(string? nameContains, C
    {
        var url = nameContains is null ? "/devices" : $"/devices?nameContains={Uri.Escap
        var devices = await _http.GetFromJsonAsync<List<DeviceInfo>>(url, Json, cancella
        return devices ?? [];
    }

    public async Task<PointInfo> ReadPointAsync(int serial, string pointType, int pointI
    {
        var point = await _http.GetFromJsonAsync<PointInfo>(
            $"/devices/{serial}/points/{pointType}/{pointIndex}", Json, cancellationToke
        return point ?? throw new InvalidOperationException($"Device {serial} has no {pc
    }

    public async Task<WriteResult> WritePointAsync(int serial, string pointType, int poi
    {
        var response = await _http.PutAsJsonAsync(
            $"/devices/{serial}/points/{pointType}/{pointIndex}", new { value }, Json, c
        response.EnsureSuccessStatusCode();
        var result = await response.Content.ReadFromJsonAsync<WriteResult>(Json, cancell
        return result ?? new WriteResult(false, "The service returned no answer.");
    }
}
```

Which service answers on `TEMCO_PLATFORM_API` is open decision 1 in §11. Only `HttpPlatformApi` is affected by the answer.

5.6 The catalog database, read-only

`temco-store` owns the file; the server only reads it.

CSHARP

```csharp
using Microsoft.Data.Sqlite;

namespace Temco.Mcp;

public sealed class CatalogDb
{
    private readonly string _connectionString;

    public CatalogDb(string path)
    {
        // The extension opened this file with journal_mode=WAL, so a reader here never
        _connectionString = new SqliteConnectionStringBuilder
        {
            DataSource = path,
            Mode = SqliteOpenMode.ReadOnly,
            Pooling = true,
        }.ToString();
    }

    public async Task<IReadOnlyList<IReadOnlyDictionary<string, object?>>> QueryAsync(
        string sql, CancellationToken cancellationToken)
    {
        await using var connection = new SqliteConnection(_connectionString);
        await connection.OpenAsync(cancellationToken);
        await using (var pragma = connection.CreateCommand())
        {
            // Read-only, but still a shared file: wait rather than fail while a writer
            pragma.CommandText = "PRAGMA busy_timeout = 5000;";
            await pragma.ExecuteNonQueryAsync(cancellationToken);
        }
        await using var command = connection.CreateCommand();
        command.CommandText = sql;
        await using var reader = await command.ExecuteReaderAsync(cancellationToken);
        var rows = new List<IReadOnlyDictionary<string, object?>>();
        while (await reader.ReadAsync(cancellationToken))
        {
            var row = new Dictionary<string, object?>(StringComparer.Ordinal);
            for (var i = 0; i < reader.FieldCount; i++)
            {
                row[reader.GetName(i)] = await reader.IsDBNullAsync(i, cancellationToken
```

```
            }
            rows.Add(row);
        }
        return rows;
    }
}
```

The path comes from the extension, which uses the same expression as `catalog/store.ts:1473`: `vscode.Uri.joinPath(context.globalStorageUri, 'catalog', 'catalog.db').fsPath`.

5.7 The tool catalogue

Names are the contract; the model sees them verbatim. Twenty-three is deliberately short — a local 7B model loses accuracy as the list grows and the arguments deepen — and every argument is a scalar.

Tool	Reads / writes	Backed by
`device.list`	reads devices	`IPlatformApi.GetDevicesAsync`
`device.describe`	reads one device	`IPlatformApi`
`device.read_point`	reads one point	`IPlatformApi.ReadPointAsync`
`device.write_point`	writes one point	`IPlatformApi.WritePointAsync`
`device.trendlog_list`	reads trendlog names	`IPlatformApi`
`device.trendlog_query`	reads history over a range	`IPlatformApi`
`device.alarm_list`	reads alarms, optionally active only	`IPlatformApi`
`device.alarm_acknowledge`	acknowledges one alarm	`IPlatformApi`
`docs.search`	full-text over the published documentation	`docs/` in the repository
`docs.read`	reads one page by path	`docs/`
`docs.outline`	reads the table of contents	`docs/README.md`
`docs.stale`	counts pages whose code moved	`docs/`
`repo.list`	reads repositories	`IPlatformApi`
`repo.show`	reads one repository	`IPlatformApi`
`repo.status`	reads the working tree of one clone	`IPlatformApi`
`repo.log`	reads commits	`IPlatformApi`
`repo.diff`	reads a diff; never applies one	`IPlatformApi`
`project.list`	reads projects	`IPlatformApi`
`project.create`	creates one project	`IPlatformApi`
`catalog.query`	runs one read-only statement	`CatalogDb`
`catalog.tables`	lists tables and columns	`CatalogDb`
`learn.search`	searches the site's content	`IPlatformApi`
`learn.post_status`	reads the publication state of a topic	`IPlatformApi`

5.8 Audit

One row per call, in the extension's storage, so what the assistant did can be read back.

CSHARP

```csharp
using Microsoft.Data.Sqlite;

namespace Temco.Mcp;

public sealed class Audit
{
    private readonly string _path;

    public Audit(string appDataDirectory)
    {
        _path = Path.Combine(appDataDirectory, "temco-mcp", "audit.db");
        Directory.CreateDirectory(Path.GetDirectoryName(_path)!);
        using var connection = new SqliteConnection($"Data Source={_path}");
        connection.Open();
        using var command = connection.CreateCommand();
        command.CommandText = """
            CREATE TABLE IF NOT EXISTS calls (
                at TEXT NOT NULL,
                tool TEXT NOT NULL,
                arguments TEXT NOT NULL,
                rows INTEGER NOT NULL
            );
            """;
        command.ExecuteNonQuery();
    }

    public void Record(string tool, string arguments, int rows)
    {
        using var connection = new SqliteConnection($"Data Source={_path}");
        connection.Open();
        using var command = connection.CreateCommand();
        command.CommandText = """
            INSERT INTO calls (at, tool, arguments, rows) VALUES ($at, $tool, $arguments
            """;
        command.Parameters.AddWithValue("$at", DateTimeOffset.UtcNow.ToString("O"));
        command.Parameters.AddWithValue("$tool", tool);
        command.Parameters.AddWithValue("$arguments", arguments);
        command.Parameters.AddWithValue("$rows", rows);
        command.ExecuteNonQuery();
```

```
    }
}
```

The policy gate is in the loop (§6.4), not in the tool: a call to something in the `DESTRUCTIVE` set is only made after the panel asks the person.

---

6. The client inside `temco-ai`

6.1 Nothing to install

The client is written in `ai/mcp.ts` rather than pulled in as a dependency. `@modelcontextprotocol/client` would be the first runtime dependency in an extension that installs nothing, and this extension is compiled with `tsc` rather than bundled, so a dependency would also have to be packaged and kept in step with every release. The protocol the assistant needs is four messages long, and the server is ours.

`temco-ai` therefore keeps `"dependencies": {}`. The version that used the SDK typechecks, but it would have been the only reason this extension needed a `node_modules`.

6.2 `ai/mcp.ts`

Two functions over two messages, section 8.4:

Export	What it does
`fetchTools(endpoint, token, log)`	Posts `tools/list` and answers with `ToolSpec[]` built from each tool's `name`, `title`, `inputSchema` and annotations
`runTool(endpoint, token, name, args, token)`	Posts `tools/call` and joins the `content[].text` it gets back. A failed call is not special-cased: the server already returns its reason as text, and that text is what the model should read

Both go through one `post()`: an `http.request` to the endpoint with `Authorization: Bearer <token>`, `Accept: application/json, text/event-stream`, and the answer read to the end. The transport frames the answer either way — the ASP.NET implementation streams it as a server-sent event with one `data:` line — so `unwrap()` takes the `data:` lines when the content type says `text/event-stream` and the whole body otherwise, then reads `result` or throws the server's `error.message`.

There is no session to open and nothing to dispose: the server runs in its default stateless mode, so each POST carries everything and a request that arrives while another is in flight is unaffected. That is why the client has no class any more — the two things that made one necessary, a pipe to write to and a handshake to replay, are gone.

Timeouts are 15 s for the list and 30 s per call, both enforced by the socket's own `timeout`, and a `CancellationToken` destroys the request it belongs to. A `401` is reported as it is: the token belongs to the window's process, and a token the server refuses means the process was restarted behind the caller's back.

6.3 `ai/binPath.ts` — where the binary is

The server is an installed artifact, exactly like the runtime and the models: a catalog entry, a checksum, a folder under `modelRoot(context)`, recorded in the manifest by `install()`.

TYPESCRIPT

```typescript
// Where the tool server is, and what it needs to know.
//
// The server is an installed artifact, exactly like the runtime and the models: a catal
// a folder under `modelRoot(context)`. Nothing here hard-codes an install path, so `tem
// moves the server with everything else.

import * as vscode from 'vscode';
import { CatalogEntry, toolServerFor } from './catalog';
import { install, installedArtifacts } from './install';

/** The catalog entry for this machine, if the catalogue has one for it. */
export function serverEntry(): CatalogEntry | undefined {
  return toolServerFor(process.platform, process.arch);
}

/** The installed executable, or `undefined` when it has not been installed on this mach
export function serverBinary(context: vscode.ExtensionContext): string | undefined {
  const entry = serverEntry();
  if (!entry) {
    return undefined;
  }
  return installedArtifacts(context).find((artifact) => artifact.id === entry.id)?.path;
}

/** Installs it if it is missing, and answers with the executable path. */
export async function ensureServerBinary(context: vscode.ExtensionContext): Promise<stri
  const entry = serverEntry();
  if (!entry) {
    throw new Error(
      `No tool server is published for ${process.platform}-${process.arch}, so the assis
    );
  }
  const artifact = await install(context, entry);
  return artifact.path;
}

/** What the server is told about this machine. Nothing else is inherited. */
export function serverEnv(context: vscode.ExtensionContext): Record<string, string> {
  const env: Record<string, string> = {};
  for (const key of ['PATH', 'HOME', 'USERPROFILE', 'SystemRoot', 'TEMP', 'TMP', 'TMPDIR
```

```
    const value = process.env[key];
    if (typeof value === 'string') {
      env[key] = value;
    }
  }
  env.TEMCO_CATALOG_DB = vscode.Uri.joinPath(
    context.globalStorageUri,
    'catalog',
    'catalog.db'
  ).fsPath;
  env.TEMCO_MCP_STATE = context.globalStorageUri.fsPath;
  env.TEMCO_PLATFORM_API = platformApi();
  return env;
}

/** The address the device, repository and project tools ask. Empty means they are not u
export function platformApi(): string {
  return vscode.workspace.getConfiguration('temcoAi').get<string>('platformApi', '').tri
}
```

Three consequences, all of them wanted:

- The installer stays the size it is; the server arrives on first use, the same way a model does, and asks once before it does.
- `ArtifactKind` in `catalog.ts:7` becomes `'runtime' | 'model' | 'tool-server'`, and one `install()` serves all three. `modelRoot(context)` stays the single root, so `temcoAi.modelFolder` moves the server too.
- A missing binary is an ordinary state, not an error: the Tool Server page's **Start** downloads and starts it, and until then the plain answer path runs unchanged.

6.4 `ai/agent.ts`, the loop

The loop keeps its shape — ask, run what it asked for, ask again, with a six-step budget and a set of calls that need a person's yes. What changed is where the server comes from: it is not opened and disposed per turn any more, because it is a process the window owns and the Tool Server page shows.

TYPESCRIPT

```typescript
import { platformApi, serverBinary } from './binPath';
import { ChatMessage, ToolSpec, complete } from './runtime';
import { callThroughServer, toolServer } from './server';

/** The tools the server offers, without asking the model anything. Starts the server wh
export async function listTools(context: vscode.ExtensionContext): Promise<readonly Tool
  return toolServer().tools(context);
}

export async function callTool(
  context: vscode.ExtensionContext,
  name: string,
  args: Record<string, unknown>,
  options: { log?: (line: string) => void; approve?: Approval; token?: vscode.Cancellati
): Promise<string> {
  if (needsApproval(name)) {
    const yes = options.approve ? await options.approve(name, args) : false;
    if (!yes) {
      options.log?.(`[mcp] ${name} declined`);
      return 'The person at the keyboard declined this call.';
    }
  }
  return callThroughServer(context, name, args, options.token);
}

export async function runAgent(context, messages, options): Promise<AgentResult> {
  // No platform service is no longer fatal: the catalog tools answer from this machine.
  if (platformApi().length === 0) {
    options.log('[mcp] no platform service is configured (temcoAi.platformApi); tools th
  }

  const tools = await listTools(context);
  for (let step = 0; step < MAX_STEPS; step++) {
    const reply = await complete(context, thread, { tools, token: options.token });
    // ... unchanged: push the assistant turn, gate each call, run it through callThroug
  }
}
```

Three properties are worth naming, because each was a decision:

- **Both entry points still pass one gate.** `callTool()` asks before a write, and `runAgent` asks before a write, and both are in this file so a third caller cannot arrive without one.
- **The approval set stays in the extension.** It is not read from the server's `destructiveHint`, which a tampered server could set to `false`.
- **A missing platform service is a sentence, not a throw.** It used to stop the turn; now the catalog tools answer and the rest say the service did not answer, which is something the model can pass on to a person.

6.5 Lifecycle

`ai/server.ts` owns the process. It is the one place that starts it, so the panel, the Tools page's runner and the Tool Server page all talk to the same thing.

Event	What happens
Nothing needs the server	It is not running. Opening a page never starts it, and `status()` says `stopped`
A turn, or the Tools page, needs the catalogue	`toolServer().tools(context)` calls `ensure()`, which starts it if it is not up and reuses the process if it is
First start on a machine with no server	`start()` downloads it through `install()`, then starts it — "Start" is the button that installs
Port	`temcoAi.serverPort`, or a free port found by binding one and releasing it. Never a fixed default: two windows must not collide
Token	24 random bytes per start, passed in `TEMCO_MCP_TOKEN`, required on every request but `/health`. It is not a secret to be stored: it dies with the process
Readiness	`/health` is polled every 250 ms for up to 30 s. Only a 200 counts; a process that is up but silent is not running
The process exits on its own	State `failed`, with the last three lines it wrote, so the page can say what happened instead of "stopped"
`temcoAi.serverIdleMinutes`	`0` (the default) keeps it for the window. Anything else stops it after that long without a call
The window closes	`deactivate()` calls `disposeToolServer()`, which kills the child. Nothing is left behind
No artifact for this platform	`serverEntry()` is undefined, `start()` fails with that sentence, and the plain answer path still runs

7. Building it

7.1 Locally

```powershell
winget install Microsoft.DotNet.SDK.10   # once per machine; .NET 10 is the current LTS

dotnet publish tools/temco-mcp/Temco.Mcp.csproj -c Release -r win-x64 --self-contained t
  -p:PublishSingleFile=false -o artifacts/temco-mcp-win-x64
dotnet publish tools/temco-mcp/Temco.Mcp.csproj -c Release -r linux-x64 --self-containec
  -p:PublishSingleFile=false -o artifacts/temco-mcp-linux-x64
dotnet publish tools/temco-mcp/Temco.Mcp.csproj -c Release -r osx-arm64 --self-containec
  -p:PublishSingleFile=false -o artifacts/temco-mcp-osx-arm64
```

The three RIDs are the three rows of the release matrix in `.github/workflows/release.yml`:

Release matrix	`process.platform` / `process.arch`	RID	Catalog entry id
`win32` / `x64`	`win32` / `x64`	`win-x64`	`temco-mcp-win-x64`
`linux` / `x64`	`linux` / `x64`	`linux-x64`	`temco-mcp-linux-x64`
`darwin` / `arm64`	`darwin` / `arm64`	`osx-arm64`	`temco-mcp-osx-arm64`

`serverEntry()` matches on `process.platform` and `process.arch`, so the table is the only place the mapping is written down; the folder name comes from the entry id, which `install()` uses as the unpack folder.

7.2 What it produces

344 files and 108 MB for one platform, because a self-contained publish carries the runtime with it — and, since the HTTP transport arrived, ASP.NET Core with it. The whole folder is the artifact: apphost, `temco-mcp.dll`, `e_sqlite3.dll`, and the framework. Nothing is written next to it at run time, so it may sit in a read-only store.

7.3 Into an archive, and its checksum

`install()` unpacks with the platform's own `tar`, which reads `.tar.gz` everywhere (GNU tar does not read `.zip`), then finds the executable by name anywhere inside. The archive is the unit that is downloaded and verified, exactly as a model is:

```powershell
tar -czf artifacts/temco-mcp-win-x64.tar.gz -C artifacts temco-mcp-win-x64
(Get-FileHash artifacts/temco-mcp-win-x64.tar.gz -Algorithm SHA256).Hash.ToLowerInvariar
```

The hash and the byte count go into the `TOOL_SERVERS` entry in `catalog.ts`, with `kind: 'tool-server'`, `platform`, `architecture` and `licence`. **They are not pinned yet**, and until they are, `install()` asks once before it downloads: a modal naming the file, the exact URL, the size (or "not published in the catalog") and the folder it lands in, with one **Download** button. The checksum the file turns out to have is recorded in the manifest either way, so a later change is still noticed. `temcoAi.allowUnpinned` skips the question for a machine that installs unattended — it no longer *is* the question. The unpacked folder `temco-mcp-win-x64` is what `install()` records, and `artifact.path` is the executable inside it.

That is the whole first run, and it was exercised end to end against a local HTTP server serving the real archive: cancel installs nothing; **Download** fetches 49,248,480 bytes, writes the manifest (`id, kind, file, sha256, bytes, url, path, installedAt`) and leaves `temco-mcp.exe` where `serverBinary()` looks; a second install asks nothing; `allowUnpinned` asks nothing.

`.gitignore` carries `tools/temco-mcp/bin/`, `tools/temco-mcp/obj/` and `extensions/temco-ai/bin/`: build output of the machine, never of git.

7.4 In CI

In `.github/workflows/release.yml`, in the build job. The SDK is installed next to Node, and the publish happens after the application is packaged and before the platform's staging step, so the archive lands in the same `staging/` folder the release collects:

YAML

```yaml
      - uses: actions/setup-dotnet@v4
        with:
          dotnet-version: 10.0.x

      - name: Stage the tool server
        shell: bash
        run: |
          set -euo pipefail
          case "${{ matrix.platform }}-${{ matrix.arch }}" in
            win32-x64) rid=win-x64 ;;
            linux-x64) rid=linux-x64 ;;
            darwin-arm64) rid=osx-arm64 ;;
            *) echo "::error::No tool server is published for ${{ matrix.platform }}-${{
          esac
          dotnet publish tools/temco-mcp/Temco.Mcp.csproj -c Release -r "$rid" --self-co
            -p:PublishSingleFile=false -o "tool-server/$rid"
          mkdir -p staging
          tar -czf "staging/temco-mcp-$rid.tar.gz" -C "tool-server/$rid" .
          ls -l "staging/temco-mcp-$rid.tar.gz"
```

The existing `softprops/action-gh-release` step already collects `release/*.tar.gz`, so the three archives ride in the release next to the installers without a second upload step.

Two properties matter:

1. It runs on the same runner as the build, so the RID matches the artefact the release publishes. There is no cross-compilation and no RID guessing.
2. It is a separate step with its own exit code, so a broken tool server fails the release instead of hiding behind a later command — the failure mode that the Windows `pwsh` steps in this workflow were split to avoid.

The catalogue does not pin the tag. `TOOL_SERVER_RELEASE` in `catalog.ts` is `` `v${vscode.version.split('-')[0]}` ``, because every release attaches its own three archives: a pinned tag would send a newer application to an older build, or to a tag that never carried one.

7.5 During development

POWERSHELL

```powershell
dotnet publish tools/temco-mcp/Temco.Mcp.csproj -c Release -r win-x64 --self-contained t
  -p:PublishSingleFile=false -o extensions/temco-ai/bin/win-x64
```

That folder is the third place `serverBinary()` looks, and it is in `.gitignore`. A source checkout therefore runs the server from its own tree, before any release carries an artifact; a packaged application has it only if the installer put one there, which it does not. The three places are tried in order: what the installer recorded (`installedArtifacts()`), `<modelRoot>/<entry.id>` — which is what `temcoAi.modelFolder` moves, and where a manifest entry makes `installedArtifacts()` see a hand-placed copy — and `extensions/temco-ai/bin/<rid>` beside this extension.

The server answers two ways, and both are testable without the application:

POWERSHELL

```powershell
# stdio: one request at a time on a pipe, for an inspector or a script
npx @modelcontextprotocol/inspector extensions\temco-ai\bin\win-x64\temco-mcp.exe

# HTTP: the mode the application manages
$env:TEMCO_MCP_TRANSPORT='http'; $env:TEMCO_MCP_PORT='7391'; $env:TEMCO_MCP_TOKEN='abc'
extensions\temco-ai\bin\win-x64\temco-mcp.exe
curl.exe http://127.0.0.1:7391/health
#   -> {"ok":true,"name":"temco-mcp","version":"1.0.0"} without a token
curl.exe -X POST http://127.0.0.1:7391/mcp -H "Content-Type: application/json" `
  -H "Accept: application/json, text/event-stream" `
  -d '{"jsonrpc":"2.0","id":1,"method":"tools/list","params":{}}'
#   -> 401 without the header, and the 23 tools with -H "Authorization: Bearer abc"
```

---

8. Running it inside the application

8.1 The process tree

TEXT

```
temco-platform.exe                    the window
 └─ extension host (Node, type: commonjs)
     ├─ llama-server                   ai/runtime.ts:144    one model, loopback HTTP
     └─ temco-mcp                      ai/server.ts         one per window, loopback HTT
         └─ (HTTP) the platform service on 127.0.0.1:3210
```

Neither child starts at activation. `llama-server` starts on the first `complete()` ; `temco-mcp` starts the first time something needs the catalogue — a turn, the Tools page, or the Start button — and then stays until the idle setting or the closing window says otherwise. Both are stopped explicitly, so a closed window leaves no processes behind.

8.2 Where the binary is, per layout

Layout	Install root	Executable
Development	`<repo>/extensions/temco-ai` `globalStorage`, or `temcoAi.modelFolder`	`<root>/temco-mcp-win-x64/temco-mcp.exe`
Packaged	`<userData>/User/globalStorage/temcocontrols.temco-ai`	`<root>/temco-mcp-win-x64/temco-mcp.exe`
Source checkout	`extensions/temco-ai/bin/<rid>` beside the extension	`extensions/temco-ai/bin/win-x64/temco-mcp.exe`

The binary is *not* inside `extensions/temco-ai`, and no `.vscodeignore` question arises: it is installed by `install()`, the same call that installs a model, and resolved by `installedArtifacts()`. The install root is the same root the models use, so a user who moved `temcoAi.modelFolder` onto a larger disk moved the tool server with them.

The published file keeps its executable bit through `Compress-Archive` / `unzip`, and the manifest records the path that was verified. If a copy loses the bit, the first `open()` fails with `EACCES`; the one-time repair belongs next to the spawn:

```typescript
if (process.platform !== 'win32') {
  try {
    fs.chmodSync(binary, 0o755);
  } catch {
    // Read-only install; the file most likely already carries the bit.
  }
}
```

8.3 What the application passes in

Variable	Value	Why the server needs it
`TEMCO_CATALOG_DB`	`temco-store`'s `databasePath()`	The catalog, opened `ReadOnly`, WAL already on. Asked of the module that owns the file, not derived from this extension's own storage
`TEMCO_MCP_STATE`	`<globalStorage>`	Where `audit.db` is created
`TEMCO_PLATFORM_API`	`temcoAi.platformApi`, default `http://127.0.0.1:3210`	Where the device, repo, and project tools get their answers
`TEMCO_MCP_TRANSPORT`	`http`	Which of the two transports to serve. Absent means `stdio`
`TEMCO_MCP_PORT`	`temcoAi.serverPort`, or a free port the supervisor found	Where to listen, on `127.0.0.1` only
`TEMCO_MCP_TOKEN`	24 random bytes per start	The bearer token every request but `/health` must carry
`PATH`, `HOME` / `USERPROFILE`, `SystemRoot`, `TEMP*`, `LANG`	copied from the extension host	Nothing else is inherited; a curated list is auditable

Everything else is a setting read with `vscode.workspace.getConfiguration('temcoAi')` in `serverEnv()` and `server.ts`, never a file next to the binary. `TEMCO_MCP_LOG` is honoured if it is set (`information` puts the server's own per-call logging back); the extension leaves it alone, so the channel carries calls rather than the framework's chatter.

8.4 The wire contract

The server speaks both transports, chosen by `TEMCO_MCP_TRANSPORT`; the application uses `http`.

`stdio` — newline-delimited JSON-RPC on stdin/stdout, stderr for logs. One rule follows from a failure observed while writing this document: **the client must keep stdin open for the life of the session.** Driving the server from a file that ends immediately produced a full stderr log, executed tool calls, and *zero* responses on stdout. `StdioClientTransport` holds the pipe open; a hand-written probe must too. The server also never writes to stdout outside the protocol, which is why §5.3 sends logs to stderr.

`http` — `WebApplication` on `http://127.0.0.1:<TEMCO_MCP_PORT>`, with `WithHttpTransport()` in its default *stateless* mode:

Route	Behaviour
`GET /health`	`{"ok":true,"name":"temco-mcp","version":"1.0.0"}`. No token: it is how the supervisor knows the process is up, and it says nothing worth hiding
`GET /` and `GET /mcp`	The same facts in words: what this process is, that the protocol is `POST /mcp`, and that the application holds the token. No token, because someone who pasted the address into a browser was going to see *something*, and a bare 401 taught them nothing
`POST /mcp`	JSON-RPC. `tools/list` and `tools/call` work as the first request on a connection, because there is no session to establish
Any other path	401 unless `Authorization: Bearer <TEMCO_MCP_TOKEN>`

Three details the client depends on, each verified against the running server:

1. **No `Mcp-Session-Id`.** Stateless mode rejects the header outright (`400 … not supported in stateless mode`), so the client keeps no session state and one POST is one question.
2. **Answers arrive framed as events.** The content type is `text/event-stream` and the JSON is on one `data:` line, even for a plain request. A client that only tries `JSON.parse(body)` sees an empty answer.
3. **`/health` is not the same thing as ready.** It answers before the tool list does, which is why the supervisor polls it rather than the catalogue.

The listening socket is bound to `127.0.0.1` by name, never to `0.0.0.0`, and the token is generated per start: a loopback port is reachable by every process on the machine, including the three tools that write.

Pasting the address into a browser used to give 401 — correct, and useless. It now answers with the description above (`GET /` and `GET /mcp`, verified: 200 with the JSON, while `POST /mcp` without the header still answers 401 and with it answers the 23 tools). The Tool Server page says the same thing in one sentence, so nobody has to discover it from a status code.

8.5 Verifying it end to end

POWERSHELL

```powershell
# 1. the server answers MCP at all, without the application
npx @modelcontextprotocol/inspector <installRoot>\temco-mcp-win-x64\temco-mcp.exe
#    -> the inspector lists the tools from §5.7 and can call one

# 2. the runtime can call a tool at all. Both paths are printed by the extension on star
#    "[ai] starting <label> as <runtime.path> on 127.0.0.1:<port>" in the "Temco AI" out
Set-Content -LiteralPath probe.json -Encoding UTF8 -Value '{"model":"local","messages":[
& "<runtime.path>" --model "<model.path>" --host 127.0.0.1 --port 8099 --jinja
#    in a second window, once the model has loaded:
curl.exe http://127.0.0.1:8099/v1/chat/completions -H "Content-Type: application/json" -
#    -> a working setup answers with choices[0].message.tool_calls, not prose

# 3. the whole turn, in the application
scripts\code.bat
#    -> ask in the Temco AI panel: "read input 1 of device 1234"
#    -> the "Temco AI" output channel shows [mcp] 23 tools from …, then the call, then t
```

Step 2 decides whether `--jinja` alone is enough for the chosen model. If the reply is prose with no `tool_calls`, the model's template is the reason, and the answer is `--chat-template-file` or a model whose template the runtime already knows.

8.6 The second exposure: the same server for Copilot Chat

The server is a plain MCP server, so the application can also declare it to the language model that ships with the product — same binary, same environment, started by the editor instead of by `temco-ai`.

Two ways to declare it, and they are different products: `McpStdioServerDefinition` gives the editor its own `stdio` process (the mode is still there, and needs no token because nothing else can reach a pipe), while `McpHttpServerDefinition` points at the managed process — which is only right while it is running, and is why the definition provider below would have to be re-registered when the state changes. The `stdio` one is the simpler contract for a second consumer.

TYPESCRIPT

```
export function registerToolServer(context: vscode.ExtensionContext): vscode.Disposable
  return vscode.lm.registerMcpServerDefinitionProvider('temcoAi.tools', {
    provideMcpServerDefinitions: () => {
      const binary = serverBinary(context);
      return binary
        ? [new vscode.McpStdioServerDefinition('Temco Platform Tools', binary, [], serve
        : [];
    },
  });
}
```

`McpStdioServerDefinition(label, command, args?, env?, version?)`; `version` is what makes the editor notice a changed tool list and offer a refresh. The provider is called eagerly and must not prompt — anything interactive belongs in `resolveMcpServerDefinition`.

8.7 When it does not start

Symptom	Cause	Fix
`spawn … ENOENT`	The artifact is not installed, or the manifest has no entry for this platform	Press Start on the Tool Server page; it installs for this RID (§7.1)
`spawn … EACCES` on macOS or Linux	The executable bit was lost in a copy	The `chmod` guard in §8.2
`Failed` with `address already in use` in the log	`temcoAi.serverPort` names a port something else holds	Set it back to `0`, or pick another
`did not answer … within 30s`	The process started and never served — a port it could not bind, or a crash before `app.RunAsync()`	The three lines the page shows are the end of stderr; the whole log is behind **Show the log**
The page says `failed` with `stopped (exit 1)`	The server exited on its own; the message carries the last lines it wrote	Read them first — an ASP.NET host failure names itself
`the tool server refused this window's token`	The process was replaced while a request was in flight (a restart)	Press Restart, or just ask again: the next call starts it
`SQLITE_BUSY`	A writer holds the catalog longer than 5 s	The `busy_timeout` in §5.6; a long write in `temco-store` is the real cause
The model reads `An error occurred invoking …`	The tool threw instead of returning `CallToolResult` — including a refused connection to the platform service	§5.4 rule 1
Tools listed, calls return prose	The model answered instead of calling	Tighten the tool descriptions; the prompt's instruction to call is the other half
`tool_calls` never appear	`--jinja` is missing	§4.1, then §8.5 step 2

9. Left panel and pages

The activity-bar group stays `AI` (`temcoAiHub`, owned by `temco-workbench`); `temco-ai` contributes four group views into it and keeps its own secondary-sidebar view.

Surface	Id	Content
Activity bar, `AI`	`temcoAiHub`	Group container
- Draft	`temcoAi.draft`	The turn in progress, and what is waiting to be settled
- What it does	`temcoAi.whatItDoes`	The acts, and the pages the answers leant on
- Server	`temcoAi.server`	The model: runtime, model, and what it is told
- Tools	`temcoAi.mcp`	What the assistant can reach outside this window
Secondary side bar	`temcoAi.panel` (`Temco AI`)	The conversation
Editor page	`temcoAi.page`	Today's settings page: the runtime and its prompts
Editor page	`temcoAi.tools`	The catalogue of calls, and the box that runs one
Editor page	`temcoAi.connection`	The tool server: its state, its address, Start/Stop/Restart, the port, the platform address, the log
Editor pages	`temcoAi.actions` , `temcoAi.citations` , `temcoAi.stale`	Unchanged

The `Tools` group is the way in. Three rows: the conversation, the connection, and the calls. Two of them open pages of their own, one each.

Row	Command	What it does
Open the AI panel	`temcoAi.openPanel`	Reveals the conversation; **the same row puts it away.** A plain open toggles, because the row a reader opens a thing with is the row they reach for again. A caller that names a page never closes anything
The Tool Server	`temcoAi.connection`	The program behind every call: whether it is running, where it answers, how to stop it, and the two settings it reads
The tools it can call	`temcoAi.tools`	The catalogue itself

The title bar carries the same two ideas as buttons, so the group is useful without opening a page. The log is **not** a fourth row: it is a button on the Tool Server page, where the rest of the program's state is.

Two pages rather than one, because a status and a list are different questions and the first one is asked when something is wrong. A page that answered both put a **Start** button under a catalogue that had to be loaded before the button could be seen — which is exactly the state (nothing is running) in which the catalogue cannot load.

`temcoAi.tools` is a page, not a tree view, because a flat list of rows cannot show an argument schema. Two parts:

- **The catalogue**: tabs, one per part of the API, taken from the first word of each call's name and kept in the order the server lists them, with a count beside each: `Device 8`, `Docs 4`, `Repo 5`. The strip is a plain row of words with their counts: twelve-pixel labels in the section's own face at six hundred weight, the first letter capitalised, no background under the pointer or under the open tab, and a single hairline in the theme's primary colour along the bottom of the tab that is open. The strip wraps rather than scrolling, so no tab is ever hidden past its right edge. The calls of the open tab run across the row as cells: `device.list` over `Every T3000 device…`, `device.describe` over `One device in full…`, as many per row as fit, each description cut to three lines with the whole sentence on hover, so one long sentence cannot set the height of every row. The name and its sentence share one face — a name in monospace read as code, and these are not code. The name is a **link** that selects that call in the runner, seeds its arguments and scrolls there, which is the only thing a reader clicking a name could have wanted. The open tab shows six cells and carries the way to the rest at the right end of the strip: `Show 2 more`, which names what is held back, and `Show less` once it is out. There is no reads-and-writes split; the calls that stop for a person are named in one line under the strip instead of being a heading. Fetched through `tools/list` when the page opens — which starts the server if it is not up — so the list cannot drift from what the model is offered.
- **Run one**: the picker the links drive, the arguments as JSON seeded from the chosen tool's schema, and the answer. It calls the same `callTool()` the loop uses, so a write still asks the person first: the page is a second way in to the same gate, not around it.

When the catalogue cannot be read, the page says why **and offers the Tool Server page** rather than an install button: the reason is almost always that nothing is running, and starting it is a decision that belongs where its state is shown.

`temcoAi.connection` is the other half:

- **Status**: a word with a dot beside it — `Running`, `Starting`, `Failed`, `Stopped` — then the address it answers on (`http://127.0.0.1:<port>/mcp`), how many tools it offers, its process id, and how long it has been up. Under them **Start**, **Stop**, **Restart**, and **Show the log**. The buttons disable themselves while a start is in flight and when they are the wrong verb for the state, so nothing offers "Stop" to something that is not running.
- **The program**: installed or not, where, the platform, and the artifact id — with the `dotnet publish` line when there is nothing installed, because a source checkout is the case that has no release to download from.

Both **Where it answers** and **Platform API** were removed from this page, and each for the same reason: a page about one program should describe that program, not ask the reader to configure something they cannot yet use.

Removed	Why it was there	Where it lives now
Port field	A fixed port is only needed by something *outside* this window that has to reach a known address — an inspector, a script. Everyone else gets a free port per start, which is what stops two windows colliding	`temcoAi.serverPort`, in Settings. The address it produces is still shown, read-only, in **Status**
Platform API field	It names the HTTP service the device, repository and project tools ask. **No such service exists yet** (open decision 1), so the field asked a person to configure an address that could not answer	`temcoAi.platformApi`, in Settings; the `Temco AI: Platform API` command now opens the setting instead of a page

Four things the page does not do, each a decision:

1. **Nothing starts on open.** A status page that changes what it reports is not a status page.
2. **Nothing is guessed.** A stop and a crash both end in a process that is gone; only the exit itself separates them, which is why the failure branch carries the last lines the process wrote.
3. **The log is a button, not a row.** It is a page about one program, and the log is one of the things that program produces.
4. **The state is repainted from the module, not re-rendered.** Start, stop and restart each answer with a `status` message that the page's own script paints into the four facts it owns, so a field halfway through being typed is never thrown away.

Three older dialogs were removed in favour of these pages: a status toast that said only that the server was ready, an input box that asked for an address without saying what it was for, and a row that opened a page with no status on it.

10. Security

Concern	Rule
The model decides what to call	The loop is the boundary, not the model's goodwill: `DESTRUCTIVE` is a set in the extension, not a property read from the server
A write reaches equipment	Only after an explicit yes in the panel, and the yes is recorded with the arguments
The server is reachable from the network	It is not: the socket is bound to `127.0.0.1` by name, never `0.0.0.0`
Another process on this machine calls it	Every request but `/health` needs the bearer token this window generated at start. The token lives in memory, is passed in the child's environment rather than its arguments, and dies with the process
The catalog is modified	Opened `ReadOnly` (§5.6); writes stay in `temco-store`
A tool result is code	Results are text and are rendered as text, never evaluated
A downloaded server is tampered with	Pinned sha256 per platform where the catalogue has one; where it does not, the download is confirmed once with the file's origin and size in front of the person, and the checksum it had is recorded
Secrets	None are passed but the per-start token; the environment is a curated list, not a copy of the host's

11. Decisions

Settled, with the reason each one was settled that way:

1. **The server is HTTP, and the window manages it.** `TEMCO_MCP_TRANSPORT=http` is what the application starts, on loopback, with a per-start token. The reason is not throughput: a pipe has no address and no state, so a page could only ever say "ready" or nothing. A program that writes needs a stop button, and a stop button needs a process someone owns. `stdio` stays as the second mode, because it is what an inspector and a script want, and it is one `switch` in `Program.cs`.
2. **The server is downloaded, not bundled.** 108 MB and 344 files per platform in every installer is worse than a verified download on first use through the machinery that already installs models. A release attaches `temco-mcp-<rid>.tar.gz` for each of its three platforms, and the catalogue asks for the one that matches the running application's version. Bundling stays available for an air-gapped build by adding the artifact to the catalogue with a `file://` URL.
3. **Two pages, not one.** The catalogue and the connection were one page for a while. They are two questions asked at different times, and the state in which the catalogue cannot be read is the state

in which the connection is the only thing worth showing.

4. **The log is a button on the Tool Server page, not a row in the panel.** The panel names things a person goes to; a log is what a person goes to *after* something on a page has gone wrong.
5. **The supervisor starts the server, not the loop.** `runAgent` used to own the process because it was the only consumer. It has three now — the loop, the runner, the catalogue — and the page that shows the address has to be looking at the process the others use.
6. **A missing checksum is a question, not a refusal.** The entry used to demand a setting be found before the first download, which made "Start" a dead end on a machine that had never enabled it. The confirmation carries more information than the setting ever did — the exact URL, the size, the destination — and the observed checksum is recorded either way, so nothing is lost by asking instead of refusing. Pinning the hashes stays the goal; it is a data task that needs one build to exist first, and `allowUnpinned` now means only "do not ask".

Still open:

1. **Who answers `TEMCO_PLATFORM_API`.** The device, repo, and project tools need one HTTP surface. Either a service in the product answers it, or `IPlatformApi` gains an implementation that speaks T3000 protocols directly and the URL becomes a connection setting. Only `HttpPlatformApi` changes either way.
2. **Where the audit lives.** The server owns `audit.db` in this document, while the extension already owns the store schema. One has to be authoritative before the first release.
3. **Whether the 7B model may act by default.** The 1.5B model may not; the catalogue can carry a per-model `maxTools` and a `mayCallDestructive` flag.
4. **Whether one process should serve several windows.** The token is per window and the port is per window, so two windows run two processes. A shared process would need a shared token and a supervisor outside the extension host; nothing needs it yet.
5. **Whether `/health` should need the token.** It does not, so the supervisor can poll it before it knows anything. It exposes the name and version of a program whose binary is on the same disk.

---

12. Where the code is

Section	File
4, the runtime	`extensions/temco-ai/src/ai/runtime.ts`
5.1 to 5.8, the server	`tools/temco-mcp/**` (both transports: `Program.cs`)
6.1, 6.2, the client	`extensions/temco-ai/src/ai/mcp.ts`
6.3, where the binary is	`extensions/temco-ai/src/ai/binPath.ts`
6.4, the loop	`extensions/temco-ai/src/ai/agent.ts`
6.5, the process, its port and its token	`extensions/temco-ai/src/ai/server.ts`
7.2, the catalogue entries	`extensions/temco-ai/src/ai/catalog.ts`
7.3, unpacking a tool server	`extensions/temco-ai/src/ai/install.ts`
8.3, the environment	`extensions/temco-ai/src/ai/binPath.ts`, `extensions/temco-ai/src/ai/server.ts`
9, the groups and the two pages	`extensions/temco-ai/src/aiPages.ts`, `extensions/temco-ai/package.json`
9, the commands	`extensions/temco-ai/src/extension.ts`
the one output channel	`extensions/temco-ai/src/ai/log.ts`
7.4, the release step	`.github/workflows/release.yml`